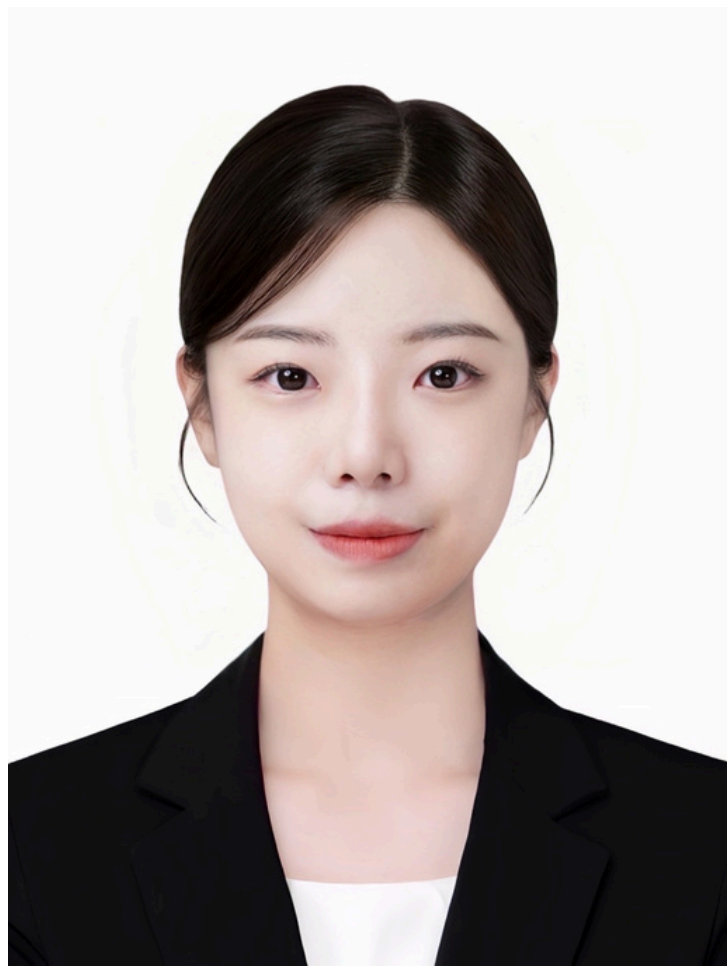




Agent가 틀릴 때, 시스템으로 바로잡는 개발자

윤경은

Agent 파이프라인 설계 · 품질 검증 · RAG 최적화 · Python 백엔드 구현

뤼튼 Agent Developer 인턴 공고를 읽고 가장 크게 와닿은 문장은 "필요한 Agentic AI를 만들기 위한 실험 과정에 참여하고 개발한다"는 부분이었습니다. 저는 이미 세 프로젝트에서 "무엇을 자동화할 것인가"보다 먼저 어디가 병목이고, 왜 이 구조가 실패하는가를 정의해 온 경험이 있습니다.

WorkFlow Agent에서는 과신하는 sLLM 응답을 4중 Guardrail과 confidence 보정으로 다뤘고, PyMate에서는 RAGAS로 병목을 측정해 임베딩과 검색 구조를 바꿨습니다. Seoul Culture Map에서는 모든 요청에 LLM을 호출하지 않도록 Intent→Retrieve→Generate 3-node 라우팅 구조를 직접 설계해 요청 비용 85%를 절감했습니다.

About

Email yge0307@gmail.com

GitHub github.com/ykgstar37-lab

Portfolio yge-portfolio.vercel.app

Background 가천대학교 응용통계학과 (2020.03 ~ 2026.08)

SK네트웍스 Family AI Camp (2025.09 ~ 2026.03)

팀 2개 · 개인 1개 AI/Agent 프로젝트

Python, FastAPI, Django 중심 구현

37% → 85%

WorkFlow Agent 정확도

LoRA v1→v3 3회 이터레이션

\$ 0.003 / req

Seoul Culture Map 요청 비용

단일 LLM 호출 대비 85%↓

0.83 → 0.97

PyMate Context Precision

RAGAS로 병목 재정의

3,468건

LoRA 학습 데이터 설계

7단계 실험 · 19건 약점 타겟팅

가장 인상 깊은 프로젝트:

TEAM

2026.02 — 2026.04

BACKEND · AI

WorkFlow Agent

GitHub

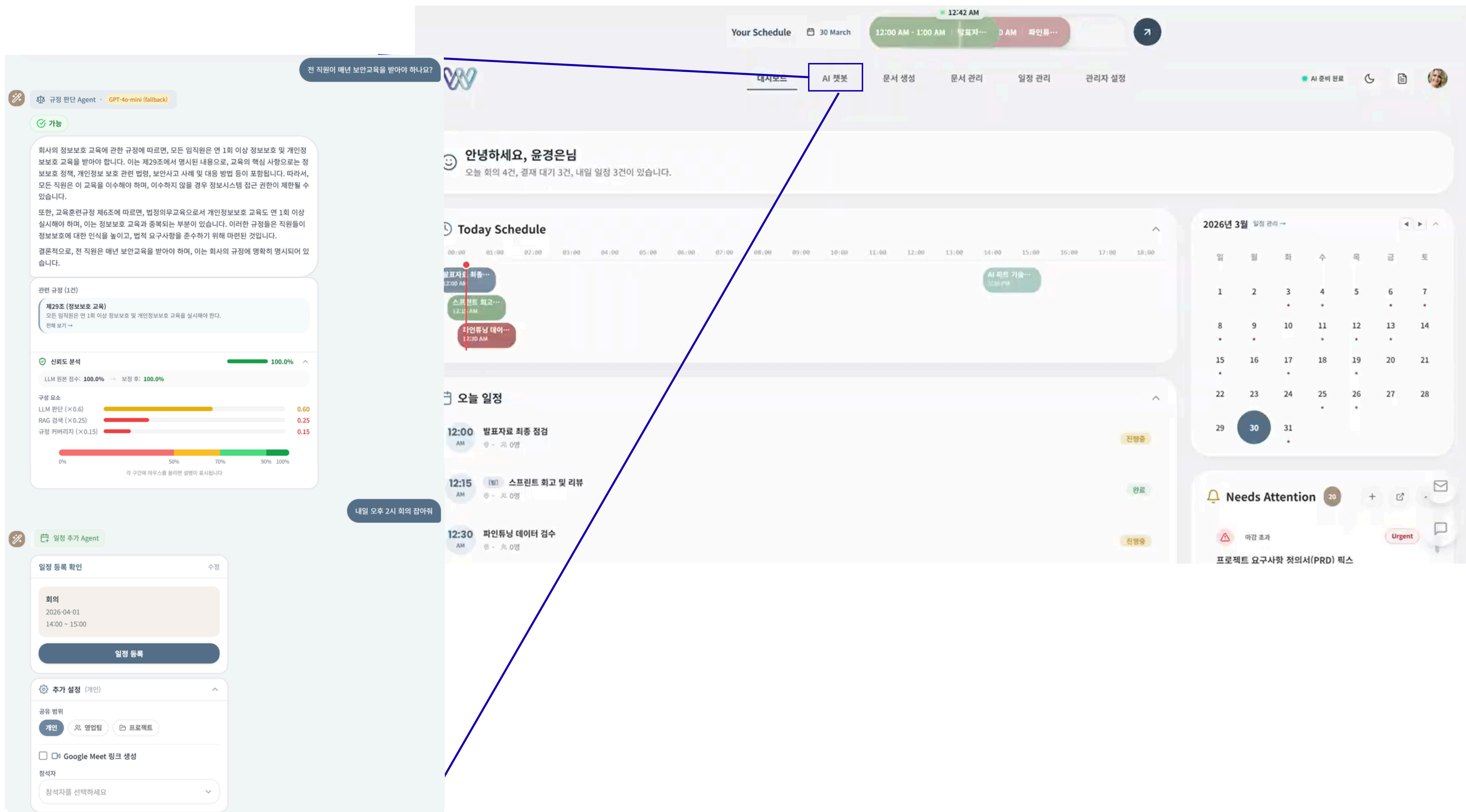
4중 Guardrail + Confidence 보정으로 과신 응답 차단

sLLM 과신 응답을 검증 구조 문제로 재정의하고, 4중 Guardrail + Confidence 보정으로 정확도 37%→85%, JSON 유효율 70%→97%를 만들었습니다.

존재하지 않는 조항을 근거처럼 말하는 답변을 서비스 레이어에서 먼저 걸러내도록 구조를 바꿨습니다.

ROLE FIT

Agent 신뢰성·Guardrail 설계·LoRA 실험 경험을 가장 가깝게 보여주는 프로젝트입니다. LLM 출력을 신뢰 대상이 아닌 검증 대상으로 다뤘고, Kanana-1.5-8B + LoRA v3 / FastAPI / LangGraph / vLLM 기반으로 설계·구현·서빙했습니다.



LangGraph

vLLM

LoRA

Guardrail

FastAPI

OpenAI API

Qdrant

PostgreSQL

Docker

AWS

GOAL

시작 배경과 목표

- 단순한 챗봇이 아니라, 실제 판단 보조에 쓸 수 있는 Agent를 만들고 싶었습니다.
- 목표는 "정답을 잘 말하는 모델"이 아니라 "틀릴 때도 위험하게 굴지 않는 시스템"을 만드는 것이었습니다.
- 따라서 정확도뿐 아니라 JSON 유효성, confidence calibration, 환각 발생 여부를 함께 봐야 한다고 정의했습니다.

ROLE - TEAM

팀 4명 분담과 내 단독 책임 영역

기여도

65%

- 팀 구성: 신지용(PM), 문지영(FE), 안혜빈(BE), 윤경은(AI)
- 내 단독 산출물: ① 판단 Agent 구조 설계
② 4중 Guardrail 구현
③ 5-factor Confidence 보장 로직
④ LoRA v1→v3 데이터셋 설계(총 3,468건)
⑤ 공통 LLM 모듈(provider 패턴) — GPT/Claude/vLLM을 1줄 교체로 전환 가능하게 통일
⑥ vLLM+RunPod A100 서버 인프라
- 공동 작업: Intent 분류 라벨링(문지영과 라벨 기준 협의), LangGraph 오케스트레이터 흐름 통합(신지용과 공동작업)

COMPARE

Guardrail 설계

AS-IS

위험한 실패: "인턴에게 AWS 접근 권한을 줘도 되나요?"
질의에 Base 모델이 오답을 내면서도
confidence 0.92를 반환.

환각 문제: 존재하지 않는 조항 번호를 근거처럼 제시해,
잘못된 응답이 더 신뢰롭게 보임.

기본 성능: 규정 판단 정확도 37.2%로 서비스 레이어에
서 바로 쓰기 어려움.

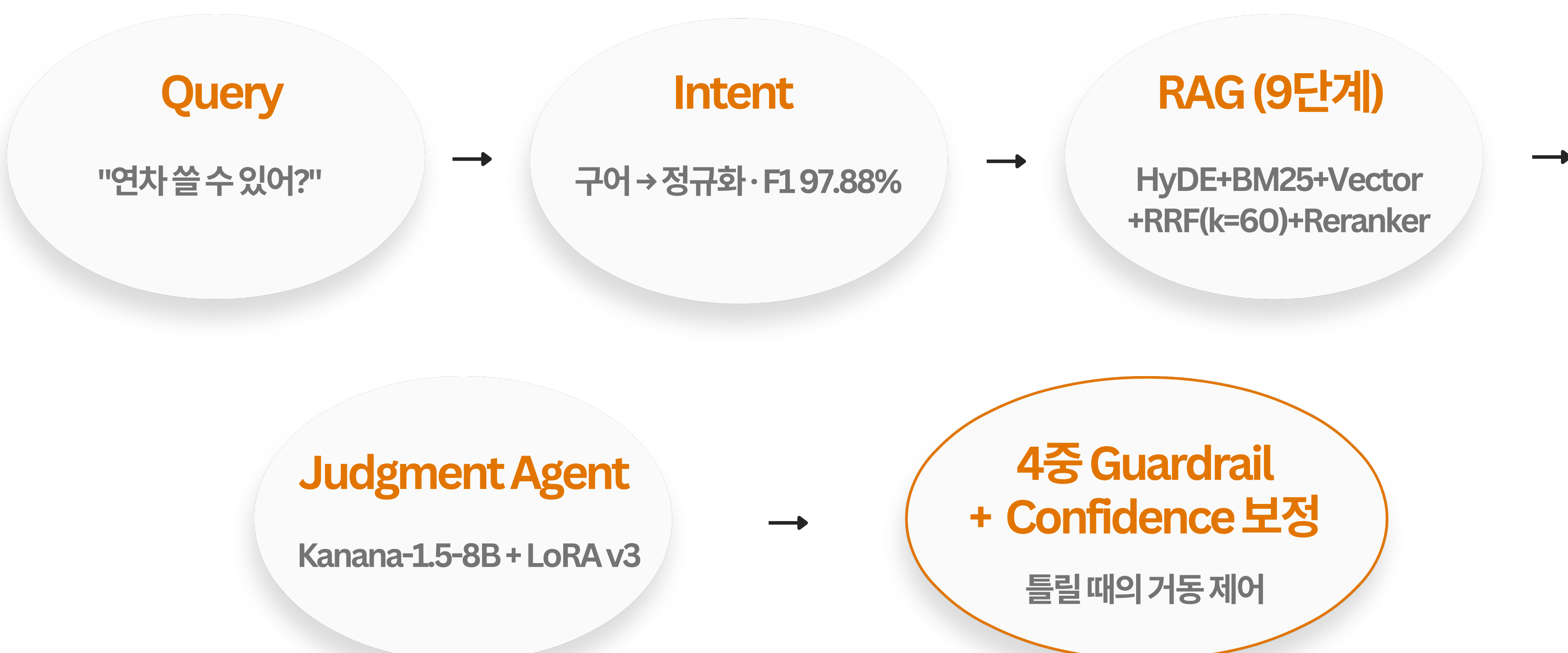
TO-BE

차단 구조: 4중 Guardrail로 키워드 매칭·조항 존재·카테고리
제한·일관성 모니터링을 분리 적용.

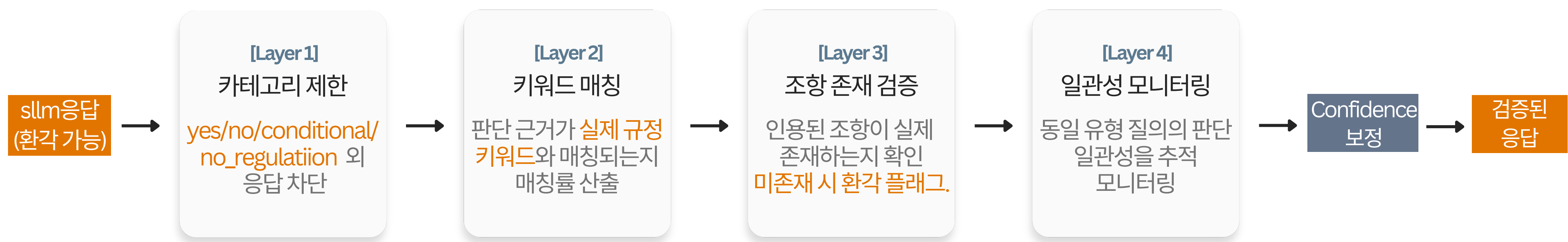
Confidence 보장: RAG 품질이 0.2 미만이면 confidence를
max 0.4로 제한해 고신뢰 오답 차단.

결과: 37%→85%. 전수 ML 검증보다 규칙 기반으로 명
확한 실패를 먼저 막음.

END-TO-END AGENT PIPELINE



4-Layer Guardrail Pipeline



위 구조는 AS-IS의 오답·환각·과신을 TO-BE의 검증 가능한 서비스 레이어로 바꾸기 위해 설계한 멀티 에이전트 + Guardrail 흐름입니다.

COMPARE

LoRA 데이터 양 vs 품질 실험

AS-IS

초기 가설: "데이터를 많이 넣으면 좋아진다"는 가정으로 v2에서 98건을 추가 보강.

실험 결과: 성능이 오히려 -3.2%p 하락해 양적 확대가 개선으로 이어지지 않음.

원인: 재량 표현이 섞인 라벨 오염 발견 — 데이터 품질 자체가 병목.

TO-BE

정밀 타겟팅: v3에서 약점 19건만 재설계 (재량 표현 14건 + 경계 케이스 5건).

판단 전환: 양 확대 대신 데이터셋 구성 자체를 바꿔, 틀린 가설을 즉시 수정.

트레이드오프: 85.4% 회복 — "더 많이"보다 "어디가 약한가"가 Agent 신뢰성의 출발점.

3단계 실험 여정

“RAG를 학습에 넣으면 좋을까?” → 실험으로 답을 찾다



v2에서 RAG 노이즈로 -9.8%p 급락 → “학습은 하드코딩(깨끗한 신호), 서빙만 RAG(실시간 검색)” 전략 확정
데이터 양 확대 가설을 폐기하고 약점 19건 정밀 타겟팅으로 전환

DECISIONS · "왜 이 숫자인가"

판단 근거

- 왜 "4중" Guardrail인가: 오답 패턴을 분류해보니 ① 질문의 핵심 키워드 누락
② 존재하지 않는 조항 인용
③ 다른 카테고리 규정을 끌어옴
④ 같은 문서 내 앞뒤 모순
— 네 종류가 전부였습니다. 3중으로는 ④번을 잡을 수 없었고, 5중부터는 직교성이 무너져 중복 감지가 발생해 4개로 고정했습니다.
- 왜 LoRA v3 데이터를 19건으로 줄였나: v2에서 98건을 추가 보강했더니 정확도가 오히려 -3.2%p 하락했습니다. 라벨 오염이 원인이었고, v2 오답 로그를 전수 분석해 conditional/규정 교차 판단 약점에 해당하는 19건만 뽑아 재설계 → +2%p 회복. "양 < 질"을 실험으로 증명한 뒤부터는 데이터 추가 전에 약점 분류를 먼저 합니다.
- 왜 RAG 컨텍스트를 학습에 안 넣었나: 초기에 RAG 결과를 학습 데이터에 섞었더니 노이즈로 -9.8%p 급락. "학습은 하드코딩된 깨끗한 데이터, 서빙은 RAG 정제 삽입"으로 분리한 뒤 안정화됐습니다.
- Label Smoothing 0.1 + 7단계 실험: confidence 0.95로 과신하는 Intent 분류 오류를 7회 반복 실험으로 좁혀 오분류 69%↓, F1 97.88% 달성.

같은 쿼리에서의 Before/After

사용자 질문 → [RAG 컨텍스트]: 정보시스템 제17조, 접근통제 제12조
인턴에게 AWS 접근 권한을 줘도 되나요?

BEFORE - BASE(37.2%)

"result": "yes" 오답 (정답: conditional)
"confidence": 0.92 근거 없이 과신
"reasoning": "접근 권한 부여가 가능합니다"
"regulations": [{"article": "제12조"}] 존재하지 않는 조항
"conditions": -
"alternatives": -
누락

AFTER - LoRA v1 (85.4%)

"result": "conditional" 정답
"confidence": 0.78 불확실성 반영
"reasoning": "수습기간 80% 이상 근무 후 제한적 접근 가능"
"regulations": [{"article": "정보보안규정 제25조"}, {"article": "인사규정 제8조"}] 실존 조항
"conditions": "보안 교육 이수 + 부서장 승인"
"alternatives": ["테스트 환경 한정 접근"]

RESULT · 산출물

정확도·안정성을 함께 끌어올린 결과

- 정확도 37%→85%, JSON 유효율 70%→97%, confidence 0.92 과신 응답을 0.78로 보정
- RAG: HyDE + BM25/Vector + RRF(k=60) + Reranker 9단계 파이프라인으로 규정 10개 문서 교차 검색
- 문서 파싱: Docling + PaddleOCR + python-docx 라우터로 PDF/DOCX/스캔 이미지 3종 지원
- FastAPI + LangGraph + vLLM + LoRA + Qdrant + Docker + AWS EC2 + RunPod A100 배포

규정 판단 정확도



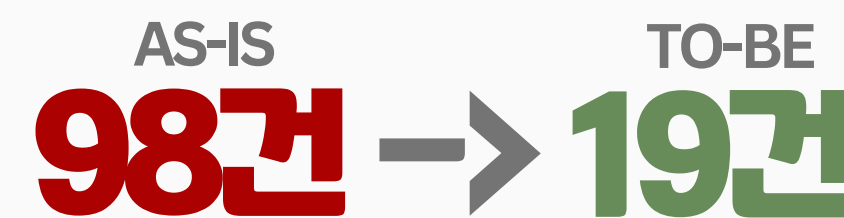
328건 사내 규정 평가셋 기준으로 정확도와 안정성을 함께 끌어올렸습니다. (+48%p)

과신 응답 보정



과신 confidence를 보정해, 틀리면서 확신하는 응답을 줄였습니다. (-0.14)

데이터 정밀 보완



데이터 양 확대보다 약점 19건 정밀 보완이 더 효과적이라는 점을 실험으로 확인했습니다. (-79건, 약 81% 감소)

RETROSPECTIVE · 아쉬운 점 / 다음에 하고 싶은 것

Agent를 신뢰의 대상이 아닌 검증의 대상으로

- **[높음]** conditional(재량적 표현) 정확도 78% — 목표 85% 대비 -7%p 미달. 경계가 모호한 케이스 (예: "필요한 경우", "원칙적으로")에서 판단이 흔들렸고, 학습 데이터를 더 수집해야 합니다.
- **[중간]** 멀티턴 미지원 — 단건 Q&A 구조로 세션 맥락이 끊깁니다. 다음 버전에서는 세션 메모리와 이전 턴 참조 구조를 추가하고 싶습니다.
- **[낮음]** Reranker가 응답에 +5.7초를 더합니다. 비동기 Reranking이나 top-k 캐싱으로 체감 속도를 개선할 여지가 남아 있습니다.
- **핵심 인사이트:** Agent 시스템은 "잘 답하는 능력"만큼 "틀릴 때의 거동"이 중요합니다. LLM 출력을 신뢰가 아닌 검증 대상으로 두고, sLLM 신뢰성을 데이터·코드·인프라 세 층에서 동시에 다루는 감각을 익혔습니다.

MORE PROJECTS

실험을 서비스로 바꾼 두 개의 프로젝트:

PERSONAL

2026.03 — 현재

AGENT ROUTING · 비용 최적화

Seoul Culture Map

GitHub

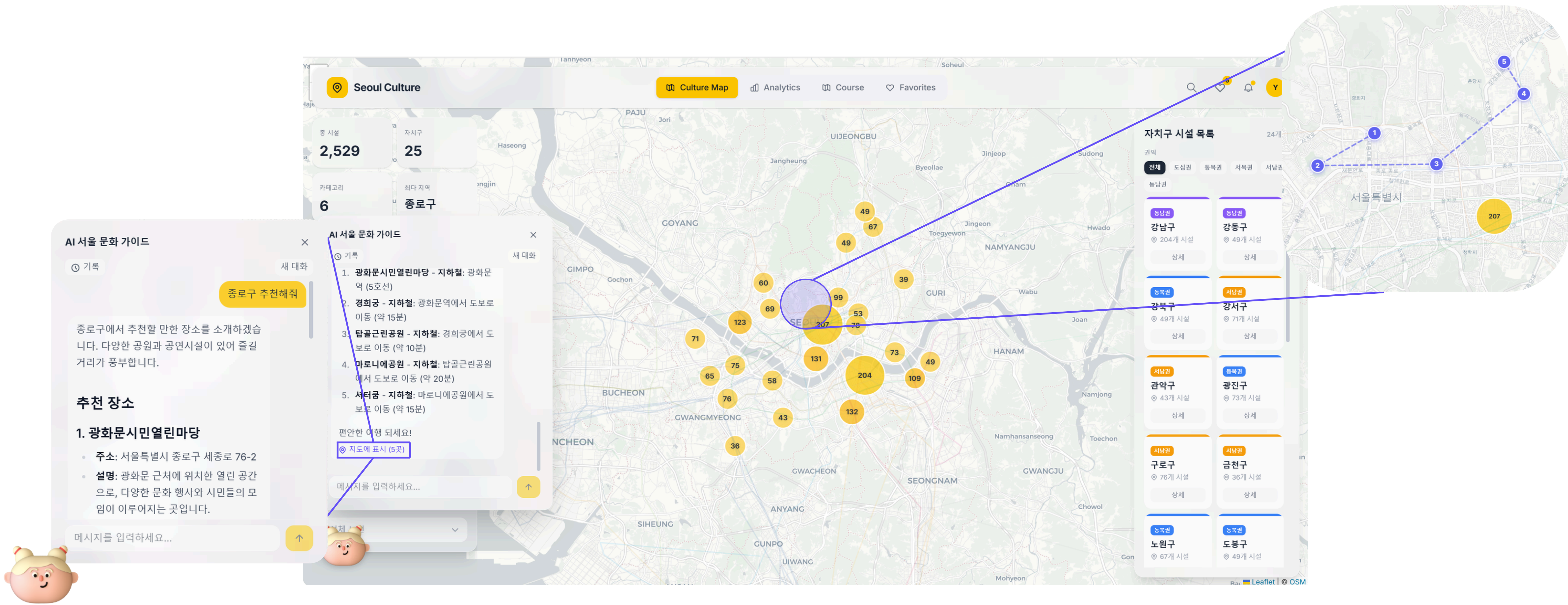
Intent 라우팅으로 요청당 LLM 호출 85% 절감

PDF로 묻히던 분석 결과를 Intent 라우팅 기반 RAG 서비스로 전환해 요청당 비용 \$0.003, 단일 LLM 호출 대비 85% 절감을 만들었습니다.

서울시 문화시설 2,500+개 탐색·추천 흐름을 개인으로 설계하고 실행 환경까지 구성했습니다.

ROLE FIT

Productivity Agent의 비용 설계와 Supporter 기억 관리 관점을 가장 가깝게 연결할 수 있는 프로젝트입니다. "언제 LLM을 호출하지 않을지"와 "어떤 맥락을 세션에 유지할지" 두 축을 직접 설계·측정했습니다.



OpenAI API LangGraph ChromaDB FastAPI SSE React

GOAL

시작 배경과 목표

- 학술제 분석 결과가 PDF로만 남아 실제 탐색 경험으로 이어지지 못했고, 모든 요청에 단일 LLM 호출을 태우는 구조는 서비스화 관점에서 비용이 과했습니다.
- 목표는 "분석 결과를 웹에서 바로 쓰게 만드는 것"과 동시에, 일상 대화와 검색형 질문을 분리해 **불필요한 LLM 호출을 줄이는 구조**를 만드는 것이었습니다.
- 그래서 서울 문화시설 2,500+개를 탐색·추천하는 챗봇을 만들되, 처음부터 Intent 라우팅 기반의 비용 최적화 구조를 전제로 설계했습니다.

COMPARE

Agent Routing — 3-node 분리

AS-IS

과한 단일 처리: 2,500+ 시설 질의를 모두 단일 LLM 호출로 처리해, 모든 요청에 전체 컨텍스트 주입.

비용 비효율: 단순 인사와 실제 검색 질의가 같은 파이프라인을 타면서 토큰 비용이 불필요하게 커짐.

품질 제어 한계: 의도 분류·검색·생성이 섞여 있어 어느 단계가 실패했는지 제어 어려움.

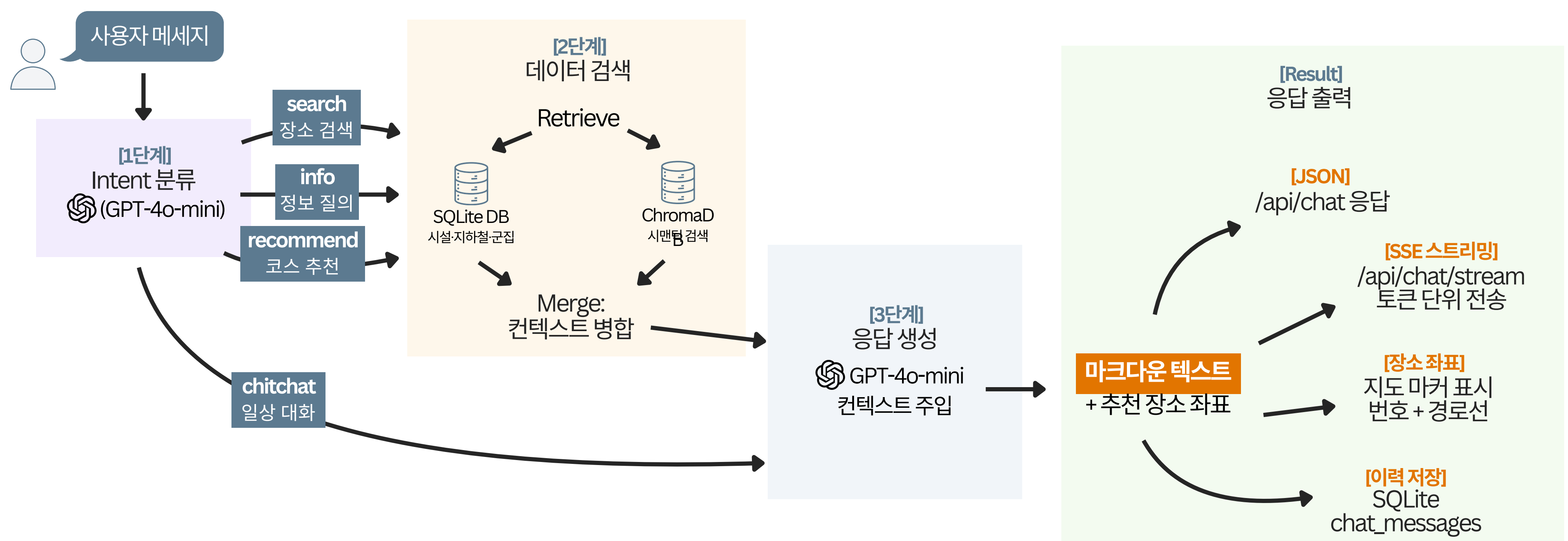
TO-BE

노드 분리: LangGraph 3-node 구조로 의도 분류·검색·응답 생성을 분리하고 각 노드를 독립 테스트 가능하게.

불필요한 호출 제거: 검색이 필요 없는 요청에는 DB/벡터 쿼리 자체를 건너뛰고, 로컬 임베딩으로 외부 API 비용도 절감.

결과: 요청당 비용 ~\$0.02→~\$0.003. "언제 검색하지 않을지"가 핵심 최적화 포인트임을 확인.

의도 분류 기반 3-Node 응답 파이프라인



Node 1 - Intent 분류

GPT-4o-mini로 3종 의도(검색·추천·일상 대화) 분류. 일상대화는 검색 노드를 스킵하여 불필요한 DB/벡터 비용 제거.

Node 2 - Retrieve

LLM 없이 SQL(필터) + ChromaDB(시맨틱) 병렬 검색. 로컬 임베딩(all-MiniLM-L6-v2)으로 API 비용 \$0.

Node 3 - Generate

검색 결과 + 세션 메모리(10턴)를 컨텍스트로 주입. SSE 토큰 스트리밍으로 첫 토큰 ~500ms

AS-IS의 단일 LLM 처리를 TO-BE의 의도 분류 → 필요한 요청만 검색·생성 흐름으로 바꾼 라우팅 구조입니다.
불필요한 LLM 호출을 줄여 비용 감소

COMPARE

컨텍스트 · 메모리 설계

AS-IS

외부 의존성: OpenAI Embedding API로 요청마다 비용 발생, 외부 서비스 의존.

UX 지연: 응답 완료까지 3-5초로 대화 흐름이 끊길 위험.

문맥 단절: 세션 문맥 미유지로 같은 질문 반복, 어떤 정보를 언제 보여줄지 설계 부족.

TO-BE

로컬 임베딩: ChromaDB + all-MiniLM-L6-v2로 API 비용을 0으로 만들고 외부 의존성 제거.

문맥 유지: 최근 10턴 대화를 SQLite에 저장하는 단기 메모리로 세션 생성·조회·삭제 관리.

응답 체감 개선: SSE 토큰 스트리밍으로 첫 토큰 ~500ms, 200건 배치·재임베딩 스킵으로 비용도 최적화.

Decisions · "왜 이 숫자인가"

판단 근거

- **로컬 임베딩 선택:** OpenAI embedding은 요청마다 비용이 누적되고 외부 의존성이 됩니다. all-MiniLM-L6-v2(384D, 33MB)를 로컬로 돌려 월 API 비용 \$0.200건 배치 처리로 메모리 부담을 줄이고, DB 대비 변화 10% 이내일 때 재임베딩을 스킵하는 로직으로 서버 재시작 시 30~60초 절약.
- **2개 공공 API 통합:** 서울 열린데이터광장은 시설 데이터가 풍부하지만 이미지가 없고, Tour API는 사진이 있지만 시설 메타가 빈약. 한쪽만 쓰면 사용자 경험이 깨졌습니다. 좌표 필드명·카테고리 코드가 완전히 달라 정규화·중복 제거 파이프라인을 직접 설계해 **2,500+ 시설 + 1,177장 이미지** 확보.
- **왜 LangGraph 3-node인가:** 단일 LLM 호출 구조로는 "검색이 필요 없는 인사 요청"에도 토큰을 소진하게 됩니다. Intent → Retrieve → Generate 3단계로 분리해 검색이 필요 없으면 LLM 호출 자체를 건너뛰도록 설계 — 요청 비용 약 85% 절감의 직접 근거.

RESULT · 산출물

비용·응답·문맥을 함께 끌어내린 결과

- 요청당 비용 **~\$0.02 → ~\$0.003** (-85%), 첫 토큰 응답 **3-5s → ~500ms**, 세션 문맥 **0턴 → 10턴** 동시 달성
- 데이터 통합: 공공 API 2종 정규화로 **시설 2,500+개 + 이미지 1,177장** 확보
- 구현 산출물: LangGraph 3-node Agent · ChromaDB + SQLite · SSE 토큰 스트리밍 · **REST API 11개 · pytest 15개** · Leaflet 지도 + K-means 군집 시각화
- FastAPI + ChromaDB + SQLite + LangGraph + SSE 기반 단일 인스턴스 서버 환경 직접 구성

요청당 비용

AS-IS TO-BE
~\$0.02 → ~\$0.003

GPT-4o-mini 토큰 단가 기준 추정치로, 불필요한 LLM 호출을 줄여 약 85% 절감했습니다.

첫 토큰 응답 시간

AS-IS TO-BE
3-5s → ~500ms

SSE 스트리밍으로 응답 완료를 기다리지 않고 첫 토큰부터 바로 보여주도록 바꿨습니다.

세션 문맥 유지

AS-IS TO-BE
0턴 → 10턴

최근 10턴 대화를 저장해 세션 내 문맥을 유지하고 반복 질문을 줄였습니다.

RETROSPECTIVE · 아쉬운 점

언제 호출하지 않을지를 고르는 감각

- **[높음]** 다국어(i18n) 미지원 — 외국인 관광객 대상 서비스인데 한국어만 구현. 다음 버전 1순위.
- **[중간]** TypeScript 미도입 — 좌표 필드명 불일치 같은 런타임 버그를 컴파일 타임에 잡을 수 있었을 것.
- **핵심 인사이트:** Agent routing은 "언제 LLM을 호출할지"를 고르는 비용 판단 문제입니다. **외부 API 의존성을 끊고 비용 곡선을 직접 통제**하는 운영 패턴(로컬 임베딩·SSE 스트리밍·10턴 단기 메모리)이 트래픽 큰 서비스의 비용 설계로 이어질 수 있다고 생각합니다.

PyMate

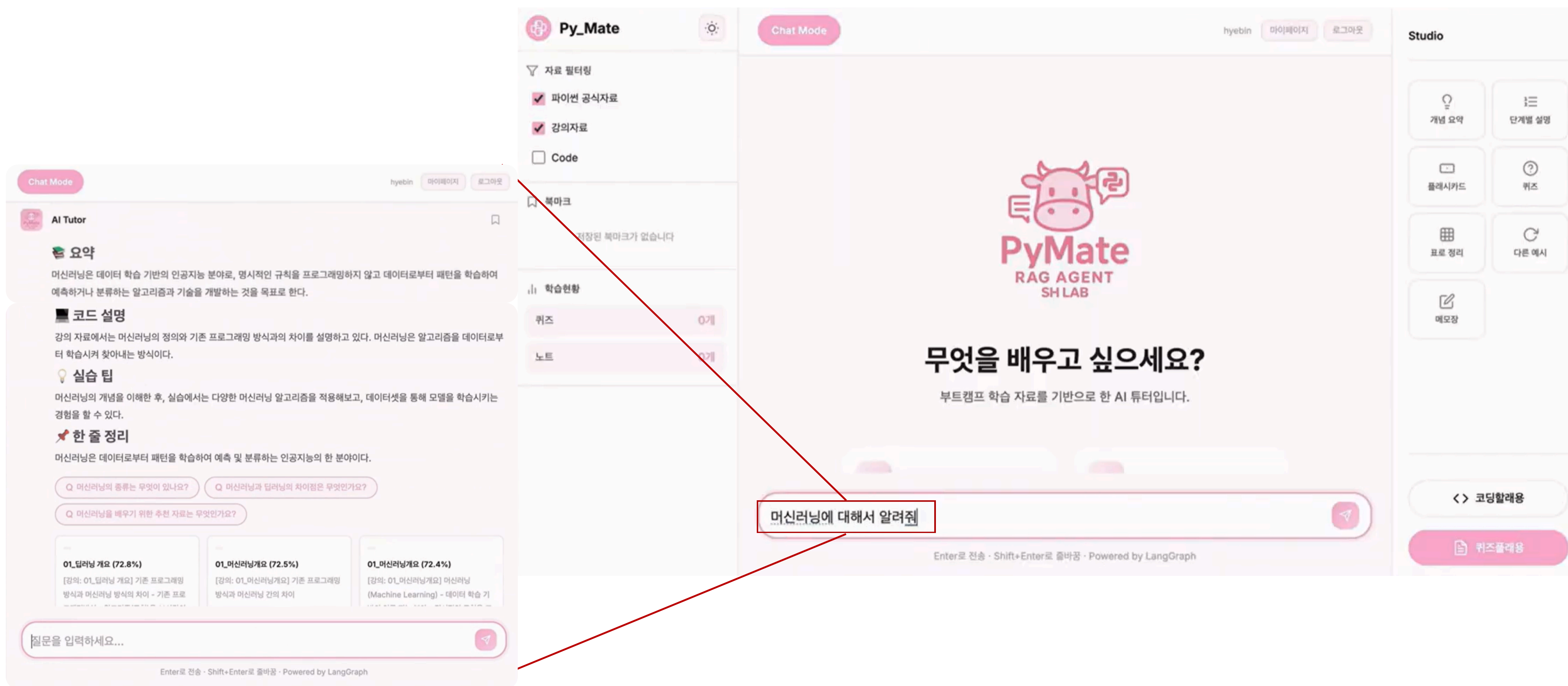
"LLM이 문제"라는 가정을 RAGAS로 반증하고 검색 구조 재설계

부정확한 답변의 원인을 RAGAS로 분리 측정해 검색 병목을 찾고, Precision 0.83→0.97, Recall 0.70→0.79로 개선했습니다.

LLM 교체보다 먼저 검색(Context Precision/Recall)과 생성 단계를 나눠 원인을 확인하는 구조를 제안했습니다.

ROLE FIT

측정 우선 · 병목 재정의 · 배포 안정화. 검색 품질을 분리 측정하고 프로덕션 전환까지 책임졌습니다.



RAGAS

Qdrant

LangChain

Django

Nginx

AWS EC2

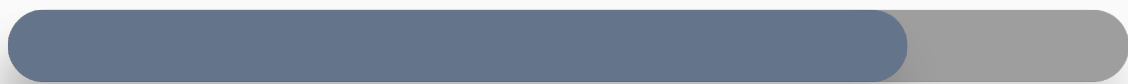
GOAL

시작 배경과 목표

- "답변이 부정확하다"는 피드백이 반복됐지만 검색 문제인지 생성 문제인지 구분되지 않았고, 팀 안에서는 LLM 교체가 먼저 거론됐습니다.
- 목표는 더 비싼 모델로 바꾸는 것이 아니라, 어느 단계가 병목인지 먼저 분리 측정해 실제 원인에 맞는 개선을 하는 것이었습니다.

ROLE - TEAM

팀 4명 분담과 내 단독 책임 영역

기여도  70%

- 팀 4명 기준 (윤경은(BE/Data·검색 품질 진단·RAG-인프라), BE 협업 1명, FE 1명, 콘텐츠/기획 1명)
- 내 단독 산출물: ① RAGAS로 검색·생성 분리 측정해 LLM이 아닌 **embedding이 병목**임을 진단
② 임베딩 **768D→3,072D 교체 의사결정**(Context Precision 0.83→0.97)

- ③ HyDE + BM25 + RRF + Reranker **9단계 RAG 파이프라인** 재설계
- ④ **Flask→Django 마이그레이션**(ORM·인증·정적파일 수동 구성을 프레임워크 기본 제공으로 정리)
- ⑤ **Nginx + Gunicorn 502 디버깅**으로 AWS 배포 환경 안정화(장애 0건 운영)

• **공동 작업:** Quiz API 3종(객관식·O/X·단답형) 설계 — 응답 스키마·채점 로직을 BE 팀원과 협의해 정리. 검색 평가·랭킹 구조·프로덕션 전환은 본인 주도.

COMPARE RAG 병목 재정의

AS-IS

문제 정의 부재: "답변이 부정확하다"는 피드백만 반복되고, 검색 문제인지 생성 문제인지 분리 측정 불가.

잘못된 대응 방향: 병목 확인 없이 바로 LLM 교체로 가자는 의견, 비용만 늘고 원인을 놓칠 위험.

제품 리스크: 관련 없는 문서가 먼저 노출되어 첫 답변 성공률이 흔들리고, 재질문 증가 가능성.

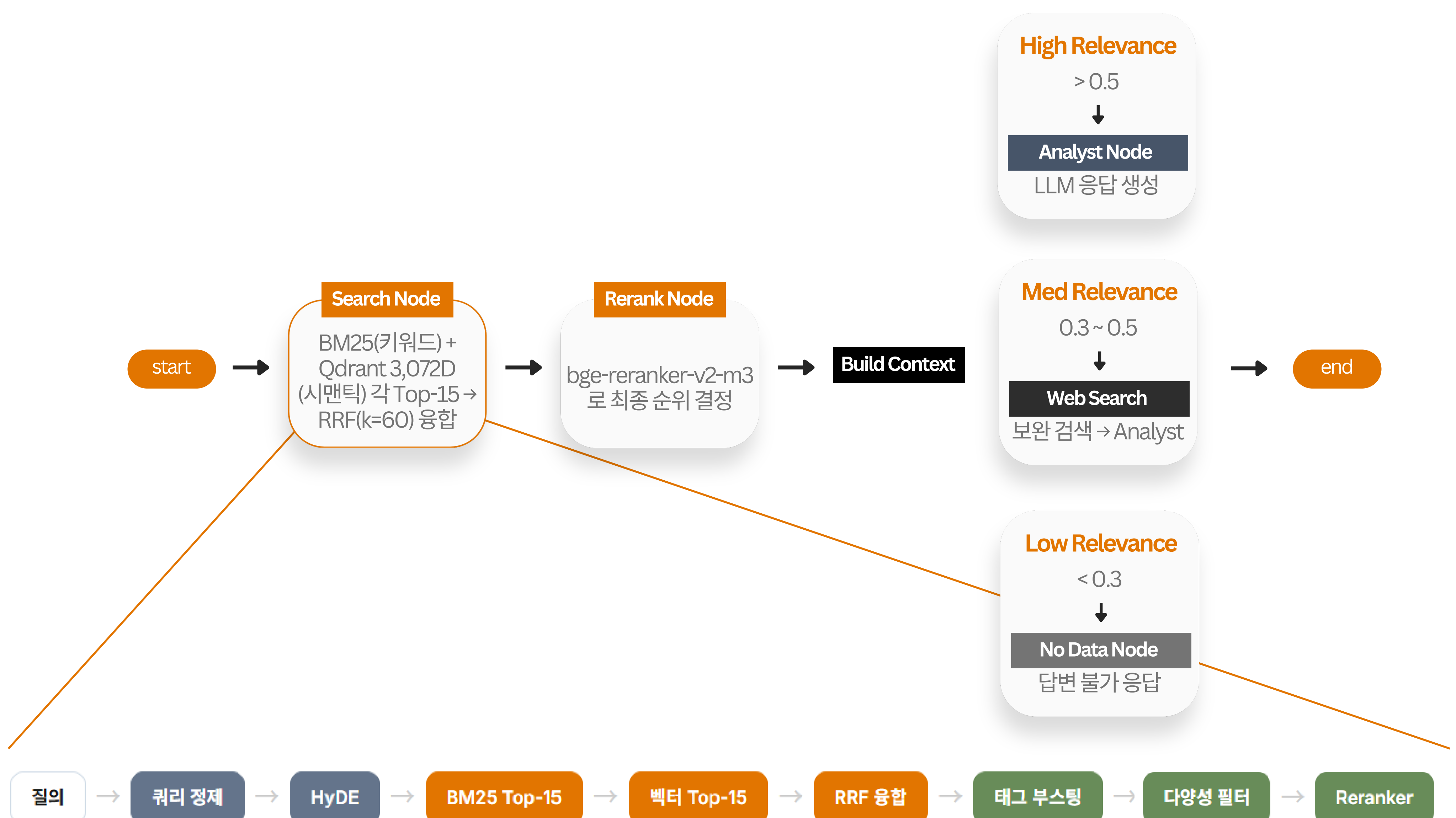
TO-BE

병목 분리: RAGAS 4개 지표로 검색·생성을 분리 측정해, Faithfulness는 높고 Precision이 낮다는 근거로 embedding 병목 특정.

해결 구조: 768D→3,072D 교체, HyDE, BM25+RRF 조합으로 짧은 질문과 긴 문서 사이의 vocabulary mismatch 해소.

결과 해석: Precision 0.83→0.97 (+14.4%p). 첫 답변 성공률과 재질문 감소의 선행지표로 해석.

RAG Workflow Flowchart



쿼리 정제 + HyDE

형태소 분석 + 동의어 확장 + 구어→문어 변환. HyDE로 가설 답변 문서를 생성하여 검색 품질 향상.

BM25 + 벡터 + RRF

BM25(키워드) + Qdrant 3,072D(시맨틱) 각 Top-15를 RRF(k=60)로 융합. 키워드·의미 검색의 상호 보완.

Reranker + RAGAS 진단

bge-reranker-v2-m3로 최종 순위 결정. RAGAS 4개 지표로 각 단계별 품질을 분리 측정하여 병목 특정.

쿼리 정제 → HyDE → BM25/Qdrant 후보 생성 → RRF 융합 → Reranker 최종 정렬

DECISIONS · "왜 이 숫자인가"

판단 근거

- **왜 임베딩 교체부터 했나:** RAGAS 측정 결과 **Context Precision 0.83**이 병목이었습니다. 생성 단계(Answer Relevancy)는 이미 양호했기 때문에 LLM 교체는 무의미하다고 판단했고, 검색 품질을 먼저 건드렸습니다.
- **왜 768D → 3,072D인가:** 1,024D·1,536D도 후보였지만 text-embedding-3-large가 한국어 교육 도메인 용어를 더 잘 잡는 걸 평가셋에서 확인. 단일 교체로 **Precision +14.4%p**.
- **왜 Reranker를 경량화했나:** bge-reranker-v2-m3는 정확하지만 응답이 무거웠습니다. ms-marco-MiniLM-L6-v2로 교체해 **약 -1초**. 정확도 손실은 평가셋에서 무시할 수준임을 미리 확인.

Deploy · "로컬은 되는데 AWS에서 502"

배포 장애 해결

- 로컬에서 잘 동작하던 코드가 AWS EC2에서 502를 반환. Nginx → Gunicorn → Django로 이어지는 소켓 경로 설정이 원인이었습니다. 요청 흐름을 단계별로 끊어 추적해 소켓 경로를 수정하고 배포 환경을 안정화했습니다.
- 이 경험이 Seoul Culture Map과 WorkFlow Agent 배포를 혼자 책임질 수 있었던 기반이 되었습니다.

RESULT

결과와 산출물

- RAGAS로 검색·생성을 분리 측정한 뒤 Precision **0.83→0.97**, Recall **0.70→0.79**로 개선했고, 병목이 LLM이 아니라 embedding임을 확인했습니다.
- HyDE + BM25 + Cross-encoder Reranker 기반 검색 구조, RAGAS 평가셋·측정 루프, Flask→Django 전환, AWS EC2 배포 안정화까지 이어지는 프로덕션 전환 경험을 만들었습니다.

관련 문서 상위 노출

AS-IS  0.83

TO-BE  0.97

관련 문서가 먼저 보이도록 랭킹 구조를 재설계했습니다. (+14%p)

검색 커버리지 확대

AS-IS  0.70

TO-BE  0.79

Recall도 함께 올려 검색 커버리지를 넓히고 누락을 줄였습니다. (+9%p)

RAG 병목 재정의

embedding

RAGAS로 검색·생성 분리 측정 후, embedding 병목 확인했습니다.

RETROSPECTIVE · 아쉬운 점

측정이 먼저였던 이유

- **[높음]** 모니터링/로깅 부족 — 프로덕션에서 어떤 질문이 실패하는지 추적하는 체계가 없어 개선 방향 설정이 어려웠습니다. 다음 프로젝트에서는 Sentry + 구조화 로그를 1일차에 넣고 싶습니다.
- **[중간]** Reranker 도입 후 응답 +1초 — 정확도 vs 속도 트레이드오프를 사용자 관점에서 A/B 검증하지 못했습니다.
- **핵심 인사이트:** "측정 먼저, 개선은 그 다음"이 이후 WorkFlow에서 LoRA 약점 타겟팅 판단까지 이어졌습니다. **RAG 품질을 숫자로 보고 인프라 병목까지 직접 다루는** 풀스택 RAG 운영 감각을 익혔고, 한 프로젝트의 실패가 다음 프로젝트의 기준이 된다는 걸 체감했습니다.

CONTRIBUTION

Agent 신뢰성, 비용 통제, 기억 관리, 운영 안정성을 각각 따로 본 것이 아니라 하나의 서비스 흐름으로 다뤘습니다. 뤼튼처럼 **Productivity Bottleneck**을 정조준하는 Agent 서비스에서는, 모델 정확도 숫자만이 아니라 틀릴 때의 거동, 호출 비용, 학습/서빙 분리까지 함께 설계하는 접근이 중요하다고 생각합니다.

뤼튼 AX CIC에서 이렇게 배우며 기여하고 싶습니다

01 할루시네이션 게이트 – 신뢰의 대상이 아닌 검증의 대상으로

뤼튼 「서포터가 잘못된 답변을 생성해요」의 잘못된 답변·긴 맥락 손실·파인튜닝 개선을, WorkFlow에서 4중 Guardrail(키워드·조항·카테고리·일관성) + Confidence 보정 + LoRA v3 약점 타겟팅으로 풀어 정확도 37%→85%, 과신뢰 오분류 69%↓를 만든 경험으로 연결하고 싶습니다. 뤼튼 Supporter 오답 패턴을 빠르게 이해하고 검증 레이어 설계에 기여하겠습니다

02 비용 통제 – Intent 라우팅으로 "안 부르는 길" 만들기

Seoul Culture Map에서 일상 대화는 Retrieve를 스킵하는 LangGraph 3-node 라우팅으로 요청당 비용 \$0.003(단일 LLM 호출 대비 85%↓)을 만들었습니다. 트래픽 큰 Productivity Agent에서도 "단순 명령 vs 추론 필요" 분기 설계가 비용 곡선을 좌우하며, baseline 측정부터 시작해 기여하고 싶습니다.

03 기억 관리 – 학습/서빙 분리 + Supporter 연결

뤼튼 「서포터 기억을 관리하는 법」의 최근 대화 자동 업데이트는 WorkFlow의 "학습/서빙 분리(RAG 컨텍스트를 학습에 넣으면 -9.8%p)" 경험과 같은 결의 문제입니다. 어떤 상태를 장기 기억으로 승격할지, 어떤 노이즈를 막을지 기준 세우는 과정에 측정 우선 원칙으로 참여하고 싶습니다.

뤼튼 AX CIC에서 Agent 신뢰성을 측정 가능한 구조로 배우고 만들고 싶습니다.

병목을 정의하고, 빠르게 실험하고, 숫자로 확인하고, 구조로 고칩니다. Agent 신뢰성·비용 통제·기억 관리·운영 안정성이 만나는 지점에서, 뤼튼 AX CIC의 문제를 가까이에서 배우며 기여하고 싶습니다.

yge0307@gmail.com · github.com/ykgstar37-lab · yge-portfolio.vercel.app